

Algorithmique (AL5)

Contrôle Continu n° 1 : Groupe Maths-Info

Nom :

Prénom :

Num. étudiant :

Groupe : B

Instructions

- Le barème est donné à titre indicatif.
- Vous pouvez utiliser tout algorithme du cours comme procédure (boîte noire), sans écrire le code.
- Quand on vous demande un algorithme pour résoudre un problème, on demande également la complexité de votre algorithme. On vise à l'optimalité de l'algorithme proposé, c'est à dire, essayer de trouver le meilleur algorithme possible pour la résolution du problème.
- Parfois il pourrait faciliter la résolution du problème de modifier le graphe donné. Dans ce cas, expliquer pourquoi l'algorithme que vous proposez sur le graphe modifié résout également le problème demandé dans le graphe original.
- Aucun document n'est autorisé.
- Durée : Une heure.

Exercice 1 : Parcours en Profondeur

- i. (3 pt) Exécuter un parcours en profondeur sur le graphe de la figure 1. Donnez les dates de pré- et post-visite pour chaque sommet, ainsi que l'arbre du parcours. Considérer que les listes d'adjacence sont données en ordre alphabétique croissant des sommets.
- ii. (1,5 pt) Y a-t-il une exécution de parcours en profondeur sur le graphe de la figure 1 telle que il n'y a pas d'arc de type retour ? Justifier votre réponse.
- iii. (1,5 pt) Le graphe de la figure 1 admet-il un tri topologique ? Si oui, donner un tri valide, sinon expliquer pourquoi.
- iv. (1,5 pt) Trouver les sources et les puits du graphe de la figure 1.
- v. (5 pt) On a déjà exécuté un parcours en profondeur sur le graphe de la figure 2 qui depart du sommet en haut à gauche. On vous donne l'arbre du parcours (les arcs du parcours sont en gras). Donnez les types des 10 autres arcs et les dates de pré- et post-visite correspondant à cet arbre de parcours.

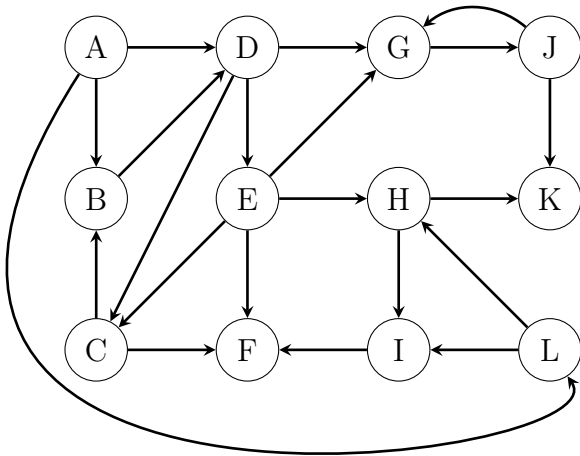


FIGURE 1 – Exécutez un parcours en profondeur.

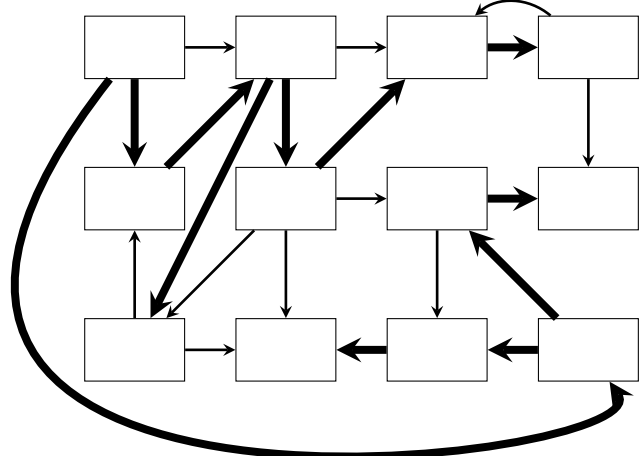


FIGURE 2 – Trouver les dates du parcours et les types des arcs.

Exercice 2 : Single Target Shortest Path (4 pt)

On est donné un graphe $G(V, E)$ orienté et pondéré sur les arcs, où les poids sont une fonction $w : E \rightarrow \mathbb{N}$ (i.e. tous les poids sont positifs). Le graphe nous est donné en forme de listes d'adjacences. Soit $t \in V$ un sommet du graphe. Trouver un algorithme pour calculer, pour tout sommet $v \in V$, les distances $d(v, t)$. La sortie de votre algorithme doit être un tableau de taille $|V|$ où la case qui correspond au sommet $v \in V$ contiendra la longueur d'un plus court chemin de v à t . Quelle est la complexité de votre algorithme ?

Exercice 3 : Multiple Sources Shortest Path (4 pt)

On est donné un graphe $G(V, E)$ non-orienté et non-pondéré. Le graphe nous est donné en forme de listes d'adjacences. Soit $s_1, s_2, \dots, s_k \in V$ k sommets du graphe (où $k \leq |V|$). Trouver un algorithme pour calculer, pour tout sommet $v \in V$, la quantité $\delta_v = \min\{d(s_1, v), d(s_2, v), \dots, d(s_k, v)\}$, où $d(s_i, v)$ est la distance entre s_i et v . La sortie de votre algorithme doit être un tableau de taille $|V|$ où la case qui correspond au sommet $v \in V$ contiendra la quantité δ_v . Quelle est la complexité de votre algorithme ? Trouver un algorithme linéaire dont la complexité ne dépend pas de k pour obtenir tous les points.

Rappels

Parcours en largeur (BFS) :

BFS(G, s):

Entree : Graphe $G(V,E)$, sommet de depart s in V

Sortie : tableau pere (arbre de parcours)

```

    creer file(Q)
    pour tout u de V faire
        marque[u]=Faux
        pere[u]=nil
    marque[s]=Vrai
    enfiler(Q,s)
    tant que Q non vide
        u=defiler(Q)
        pour tout uv de E faire
            si marque[v]=Faux faire
                marque[v]=Vrai
                pere[v] = u
                enfiler(Q,v)

```

Parcours en profondeur (DFS) :

explorer(G,u) : //la procedure de base

```

    marque[u]=Vrai
    //pre-visite si besoin
    Pour tout uv de E faire
        Si marque[v] = Faux alors
            pere[v]=u
            explorer(G,v)
    //post-visite si besoin

```

DFS(G,s): // la procedure complete

Entree : Graphe $G(V,E)$, sommet de depart s in V

Sortie : tableau pere (arbre de parcours)

```

    pour tout u de V faire
        marque[u]=Faux
        pere[u]=nil
    pour tout u de V faire
        si marque[u]=Faux faire
            explorer(G,u)
    retourne pere

```

Un résultat du cours permet de caractériser exactement le type d'un arc uv d'un graphe orienté en fonction de ces valeurs pré-visite et post-visite :

- Si $pre(u) < pre(v) < post(v) < post(u)$ alors uv est un arc avant
- Si $pre(v) < pre(u) < post(u) < post(v)$ alors uv est un arc retour
- Si $pre(v) < post(v) < pre(u) < post(u)$, alors uv est un arc transverse.

Algorithme de Dijkstra :

Dijkstra(G, s):

Entree : Graphe $G(V,E,w)$ avec une pondération $w(e)>0$ pour tout $e \in E$,

sommet de depart $s \in V$

Sortie : tableau de distances de s aux autres sommets

pour tout u de V faire

$D[u]=\text{inf}$

$\text{pere}[u]=\text{nil}$

$D[s]=0$

créer file priorite(Q)

tant que Q non vide

$u=\text{extraireMin}(Q)$

 pour tout uv de E faire

 si $D[v] > D[u] + w(uv)$

$D[v] = D[u] + w(uv)$

$\text{pere}[v]=u$

$\text{decreaseKey}(Q,v)$

retourne D

Algorithme de Bellman-Ford :

Bellman-Ford(G, s):

Entree : Graphe $G = (V,E,w)$ avec une pondération w sur les arcs,

sommet de depart $s \in V$

Sortie : tableau de distances de s aux autres sommets

$D[s]=0$

$\text{pere}[s]=\text{nil}$

pour tous les u de $V \setminus \{s\}$ faire

$D[u]=\text{inf}$

$\text{pere}[u]=\text{nil}$

repete $|V| - 1$ fois

 pour tous les $e=uv$ de E faire

 si $D[v] > D[u] + w(uv)$

$D[v] = D[u] + w(uv)$

$\text{pere}[v]=u$

retourne D